Code availability is not enough: A schema-based representation for comprehensible and extensible data analysis

H. Sherry Zhang[1], Roger D. Peng[1]

ARTICLE HISTORY
Compiled September 4, 2026

[1] Department of Statistics and Data Science, University of Texas at Austin, Texas, United States

ABSTRACT
Current reproducibility practices have greatly improved our ability to replicate published results, but they do not necessarily make data analyses easier to understand. Audience seeking to comprehend an analysis need to piece together information scattered across manuscripts and code scripts to understand the sequence of analytical steps and the rationale behind them, with some details may not explicitly record in either source. Exploring alternative analytical choices is even more challenging, as it requires understanding and modifying code that may be unfamiliar and consider how it would impact upstream and downstream code. To address these challenges, we propose a schema that represents data analyses as a collection of analytical decisions, together with their justification, inputs, and outputs. Coupled with large language models (LLMs), the schema can be inferred from natural language descriptions and translated into analysis scripts. The structured representation also enables exploration of alternative analytical choices, allowing multiple analytical workflows to be generated and compared. We demonstrate the proposed workflow using the many-analysts football study, where LLMs create the schema from the decision choices in team reports, generate corresponding analysis scripts, and explore alternative random effect specifications. As an intermediate representation between the manuscript and code scripts, our schema makes analyses more comprehensible and extendable from a reader's perspective.

KEYWORDS
Reproducibility; Large Language Models (LLMs); Analytical decisions; Many-analysts study

1. Introduction

When we want to understand a published data analysis, beyond simply being able to re-run it, there is currently nothing we can use directly for this purpose. Code and data availability, now standard practice following a broader push for reproducible research (Peng, 2011; Wrobel et al., 2024), guarantee that a result can be regenerated, but they

CONTACT: H. Sherry Zhang. Email: hsherryzhang@utexas.edu.

say nothing about whether a reader can track the decisions the authors made, understand the rationale behind them, or comfortably modify the code to explore alternative choices.

Part of the difficulty is that code and the applied researchers who write it operate at different levels: code specifies precisely what a computer should do, while a researcher reasons about what decision that computation represents in the context of their analysis, a gap also identified in prior work on authoring statistical models from conceptual relationships (Jun, Seo, et al., 2022; Jun, Birchfield, et al., 2022). Consider a line such as `dplyr::filter(df, x > 0)`, which has a precise, unambiguous meaning as code: keep rows where `x` is positive. As an analytical decision, however, that same line could represent restricting the analysis to a subpopulation for which `x > 0` is substantively meaningful, or excluding observations flagged as invalid by an encoding convention where $x \leq 0$ denotes missingness; nothing in the `filter()` syntax gives any indication of which of these two decisions is being made. Tools such as `tidyverse` (Wickham et al., 2019) have worked to align code more closely with the action being performed, naming operations after what they do rather than how they are implemented. But knowing the action a line of code performs is not the same as knowing the decision it encodes: an analysis requires additional context that the action alone does not carry.

This suggests that a more effective representation of an analysis would be one that describes the decisions made and their rationale, allowing readers to understand the analysis without needing to parse the code itself or reconstruct the rationale from the manuscript. Recent advances in Large Language Models (LLMs) for code generation make such a representation practical: LLMs are now widely used for code generation tasks, demonstrating strong capabilities in translating natural language descriptions into executable programs (Chen et al., 2021; Huang et al., 2022; Jiang et al., 2026; Lai et al., 2023; Shah et al., 2026). Once a decision is described in a schema, an LLM can generate and validate the corresponding code directly from it.

This same representation also makes extending an analysis easier. Data analysis is often underdetermined, with many plausible choices regarding data preprocessing, transformation, aggregation, and model specification (Bertran et al., 2026; Gao & Xiao, 2026; Luo et al., 2025), and a reader may reasonably want to know whether a conclusion would still hold under one of these other choices, for example, "What if we had included observations with $x \leq 0$?" Because each decision is recorded explicitly rather than embedded in code, a reader can specify such an alternative directly and have it evaluated through the same schema and the same LLM-based translation into code, rather than modifying the original script by hand.

In this paper, we present a schema designed to describe data analysis in a structured way that captures the decisions and their justifications, as well as the inputs and outputs of each step. We also demonstrate how this schema, coupled with LLMs, can be used to generate code scripts from natural-language descriptions in manuscripts, and explore alternative decision choices. We apply our schema to the many-analyst football study, where LLMs create the schema from the decision choices reported in the final reports

of each team, and generate and validate the corresponding code scripts against the original published results. In the process, we also uncover an inconsistency in one team's published report between the covariate described in the text and the one implied by their results. We further demonstrate how to the schema supports specification and exploration of alternatives random effect specification for Team 5.

The paper is organized as follows: In Section 2, we present our methodology, including the schema representation for describing data analysis and the workflow for generating and validating code from the schema by LLMs. In Section 3, we apply our methodology to the many-analyst football and demonstrate that we can reproduce the results reported in the paper, identify errors in the original reports, and highlight failure modes of current LLMs. Finally, in Section 4, we discuss the implications of our work and potential future directions.

2. Methodology

As traditional reproducibility practices can be insufficient for understanding complex modern data analyses, we have designed two complementary YAML schemas and the associated workflow to better support comprehension and extensibility in applied data analysis. Our workflow is designed to be 1) decision-centric: analyses are represented as a sequence of decisions with explicit rationale, and 2) reader-oriented exploration: readers can inspect, compare, and extend those decisions.

2.1. *A schema for describing data analysis*

We represent a data analysis, $\mathcal{S} = \{A_1, A_2, \cdots, A_n\}$, as a sequence of n analytical step A_i, applied to a dataset $\mathcal{D}$, where each step, A_i, is defined as a tuple:

$$A_i = \langle f_i, \pi_i, \rho_i, \mathcal{I}_i, \mathcal{O}_i, \omega_i \rangle. \tag{1}$$

The components of A_i can be partitioned into two parts: a required triplet (f_i, π_i, ρ_i) and an optional triplet $(\mathcal{I}_i, \mathcal{O}_i, \omega_i)$. The first component of the triplet, $f_i \in \mathcal{F}$ is the *objective* of a step, identifying the problem the step is trying to solve and thereby fixing the space of alternative decisions that could equally address it (formally introduced in Section 2.2). The second component, $\pi_i \in \Pi$, is the *decision* made to achieve f_i: it is a narration, in natural language, of what a corresponding line of code would do, situated in the context of this specific analysis, rather than the code's syntax itself. The term ρ_i is the *justification* provided for choosing the decision π_i over the other alternatives that could have fulfilled the same objective. Structuring this information into a schema involves defining the three components described above for each step. As an example, consider the description for calculating the Human Development Index (HDI) (United

Nations Development Programme, 2025):

1) Scale all four variables (life expectancy, expected years of schooling, mean years of schooling, and GNI per capita) to the [0, 1] range.
2) Combine the two education variables (expected years of schooling and mean years of schooling) into a single education variable.
3) Combine the three dimensions (health, education, and income) into a single index using a geometric mean.

In Step 3, the objective is to combine the three dimensions. While a simple arithmetic mean could be an alternative decision to fulfill this goal, the geometric mean (the decision) is chosen because it penalizes uneven development among the three dimensions (the justification). The full schema for constructing HDI is presented in Listing 1.

Listing 1 An example schema for calculating Human Development Index (HDI)

```
meta:
  type: schema
nodes:
- tag: step-scaling
  objective: how to handle variables in different scales?
  decision: apply min-max scaling to each variable
  rationale: to convert the variables to the same scale for future
    construction of a single index
- tag: step-education
  objective: how to combine the two education-related variables?
  decision: take the average
  rationale: the arithmetic mean allows perfect substitutability
    between the two variables
- tag: step-combine
  objective: how to combine the three dimensions into a single index?
  decision: use the geometric mean
  rationale: the geometric mean penalizes uneven development
    among the three core dimensions
```

The required triplet (f_i, π_i, ρ_i) is already sufficient on its own for a human reader to follow the analytical logic of a step, without needing access to the underlying dataset. The optional triplet $(\mathcal{I}_i, \mathcal{O}_i, \omega_i)$ adds a further layer on top of this: $\mathcal{I}_i$ and $\mathcal{O}_i$ bind the step to concrete variables in the dataset, resolving ahead of time the variable mapping that an LLM would otherwise have to infer at the point of code generation. Populating them requires a dataset to be available, which is why they remain optional rather than required. While the output of a step can be a pseudo object that does not yet exist in the dataset, the inputs for any given step must be within the existing data scope, comprising variables in the original dataset and all outputs from previous steps. Because the schema is meant to specify an analysis before the corresponding code is generated, an output

name can refer to a variable that does not yet exist; this is inherent to using the schema as a specification rather than a post-hoc description of existing code. In practice, $\mathcal{O}_i$ most often names a column added to the evolving dataset via non-standard evaluation, as in `tidyverse`-style pipelines; a small number of steps, such as model fitting, instead produce a distinct object that is not itself a column of $\mathcal{D}$. Formally, we require:

$$\mathcal{I}_i \subseteq \left(\mathcal{D} \cup \bigcup_{j<i} \mathcal{O}_j \right), \quad \forall i \in \{2, \ldots, n\}.$$

When the schema is extracted automatically by Large Language Models (LLMs) from text, the inputs and outputs will be inferred by the LLMs, if a dataset is provided. We also introduce a categorical confidence score, ω_i (low, medium, high), defined as the model's confidence that the recorded (f_i, π_i, ρ_i) correctly reflects the source text; ω_i is low precisely when one or more of these components could not be read off the text directly and had to be inferred by the LLM, and can be used to guide the LLM to ask for clarification to consolidate the schema extraction.

2.2. *Describing alternatives*

At each step, one decision choice is made by the researchers to address an objective, however the choice is often not the only possible one. For step i, there could be k_i decision choices that fulfill the objective f_i: $\{A_i^1, A_i^2, \ldots, A_i^{k_i}\}$. We can define a selection vector $r = (r_1, r_2, \ldots, r_n)$, where $r_i \in \{1, \ldots, k_i\}$, to describe an analysis comprised of specific series of decisions: $\mathcal{S}^r = \{A_1^{r_1}, A_2^{r_2}, \ldots, A_n^{r_n}\}$. A multiverse, used to describe the entire decision space (Simonsohn et al., 2020; Steegen et al., 2016), is defined as the Cartesian product of these sets: $\mathcal{M} = \{\mathcal{S}^r \mid r_i \in 1, \ldots, k_i, \forall i\}$. The cardinality of this space, $|\mathcal{M}| = \prod_{i=1}^{n} k_i$, quantifies the total number of unique, fully-specified analysis.

Returning to the HDI example, in Step 3, instead of a geometric mean, an arithmetic mean is another possibility to combine the three dimensions into a single index. It may not be the best decision, but still a plausible one to derive a single index from the three dimensions, hence achieve the objective of this step. Listing 2 shows the structure of an alternative YAML file. The `meta` section identifies the file type (an alternative file rather than a schema file) and specifies the analysis step and the objective under consideration. Individual alternatives are defined as entries in the alternatives list, allowing multiple alternatives to be specified within a single file.

Listing 2 An example YAML file for specifying alternative for Step 3 in calculating Human Development Index (HDI).

```
meta:
  type: alternative
  step: step-combine
  objective: how to combine the three dimensions into a single index?
alternatives:
  - id: arith-mean
    decision: use an arithmetic mean
    rationale: a simple average of the three dimensions
```

2.3. *Workflow*

The overall workflow for schema-based data analysis is illustrated in Figure 1. A schema can be created either manually from the template or automatically by an LLM from a natural-language manuscript. Once created, the schema serves as a structured specification that guides a second LLM in generating the code for the analysis. To improve reliability, our design includes a third LLM agent which is responsible for code validation. While the latest LLMs can generate reasonable code based on prompt, they do not consistently achieve perfect accuracy. The validation process runs the generated script in a sandbox environment and if an error occurs, a third LLM agent will be employed to diagnose the script. The validation agent receives the generated code together with the error message and the corresponding line number produced during the execution, and returns a newer version of the script. The revised script is then re-executed in the sandbox and this cycle continues until one of the three conditions is met: the code runs successfully, a maximum number of attempt is reached, or the user terminates the process. As will be show in Section 3, this validation process resolves the majority of R syntax issues encountered during data wrangling and statistical modelling tasks. On top of supporting a single analytical specification, the workflow also allows users to argument a schema with an alternative YAML file. This expands the original schema into a multiverse of parallel schemas, each representing a different sequence of decision choices. The same code generation and validation workflow is then applied independently to each schema, producing a corresponding set of validation scripts.

Returning to the HDI example, the proposed workflow allows both authors and readers to interact with a data analysis through natural language. The schema in Listing 1 is straightforward to construct from a template and can be included as supplementary material to document the analytical workflow. An author may use the "code generation and validation" path in Figure 1 to automatically generate the analysis script for computing HDI and validate its accuracy. A reader, on the other hand, may wish to explore how the HDI values or country rankings change if the original arithmetic mean (instead of the geometry mean) is used. By combining the original schema in Listing 1 with the alternative specification Listing 2, the "alternative generation" path produces a corresponding analysis script for each alternative with LLMs. In contrast,

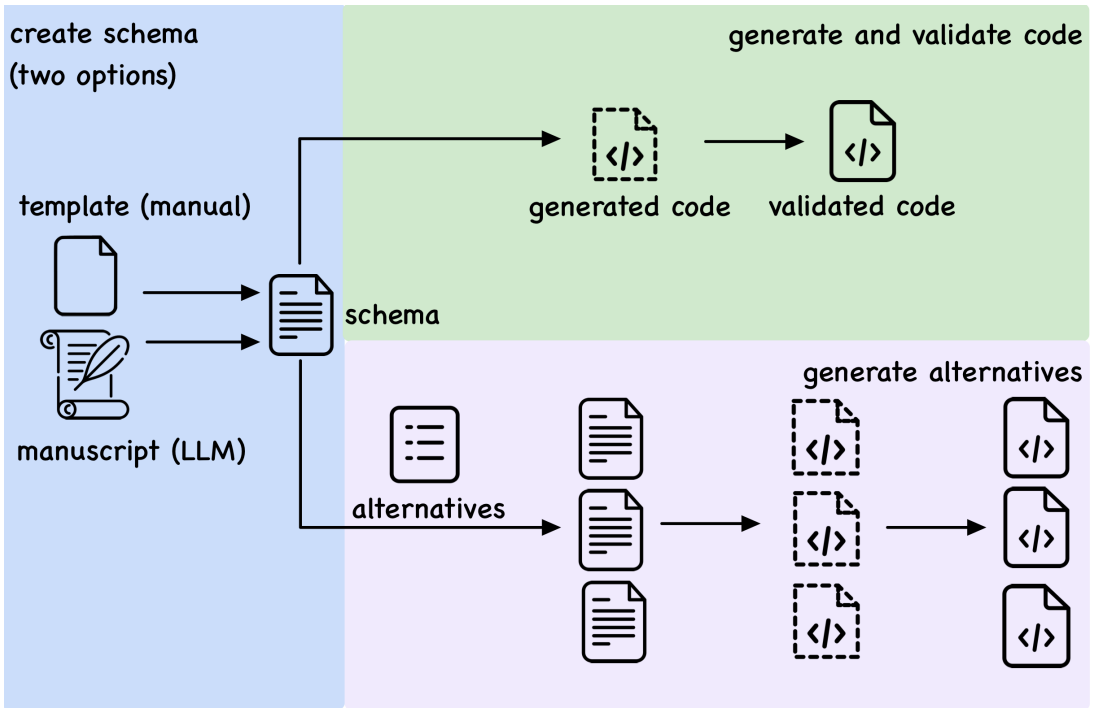


Figure 1. Overview of the schema-driven analysis workflow. Schemas can be authored manually or generated from natural language manuscripts by an LLM. Schemas are used to generate code, which is validated through iterative execution in a sandboxed environment and repaired when errors occur. Alternative specifications can expand a schema into a multiverse of parallel analyses.

modifying an existing code script by hand typically requires manually coordinating changes across upstream and downstream code. By representing analyses as a sequence of structured analytical decisions, our workflow allows LLMs to automate much of this process, making it easier to understand the sequence of steps taken in the analysis and to explore alternatives.

Several properties of this workflow follow directly from representing an analysis as an explicit schema rather than as code alone. First, the schema is *inspectable*: because it is short and structured, an analyst can check it against the source manuscript far more readily than they could audit the generated script itself, which is what makes each of the following properties useful in practice rather than only theoretical. Second, the schema *separates logic from implementation*: whether a decision was understood correctly (manuscript to schema) and whether it was implemented correctly (schema to code) become two distinct, independently checkable steps rather than a single opaque translation, a distinction Section 3 relies on directly to localize where our workflow fails. Third, the schema is *provenance-agnostic*: nothing in the workflow requires it to originate from an LLM, as Figure 1 shows, a schema can equally be authored or corrected by hand from the template, and the validation loop treats a hand-written schema no differently from an extracted one, checking it by generating and repairing code the same way. This matters because, as Section 3 shows, current LLMs do not always extract a schema from a manuscript with perfect fidelity; we do not treat this as a defect to be engineered away by further automation, but as the reason the schema is kept explicit and available for an analyst to correct, rather than folded silently into the code-generation step. Fourth, the schema is *reusable*: because a decision is represented independently of any one downstream implementation, the same schema can be systematically varied, via the alternative specification described above, to generate and compare multiple analyses without hand-coordinating changes across an existing script.

3. Application: many-analysts football study

In this section, we apply our methodology to the many-analyst football by Silberzahn et al. (2018) to reproduce the reported estimated effect size by each team. During this process, we identify some errors in the original reports and highlight failure modes of current LLMs.

Silberzahn et al. (2018) invites 29 research groups to conduct the same data analysis to answer whether are more likely to give red cards to dark-skin-toned player than to light-skin-toned players. The provided dataset contains two raters' skin color rating to players' picture, the aggregated number of red card received by a player from a referee across all matches in that player-referee pair, and other relevant player, referee, and game characteristics: player birthday, height, weight, player position, league country, number of victories, ties, and defeats, and number of yellow and red cards received. In Silberzahn et al. (2018), each group first conducted an initial analysis separately, followed by a round-robin peer evaluation. Each group was then given the opportunity

to incorporate feedback from the discussion and submitted a final analysis as a report with a formal Method section. All the materials related to the study is publicly available on the Open Science Framework (OSF).

Based on the reports presented by each team, most participating groups adopted a modelling approach to estimate the effect of skin color rating on the likelihood of receiving a red card. As an illustrative example, Group 5 first transformed the data into player–game observations, resulting in a binary response variable for whether a player received a red card, and removed observations with missing values using list-wise deletion. The resulting data were modelled using a generalized linear mixed model (GLMM) with a binomial error distribution. The average skin color rating from two raters was used as the primary covariate, and its estimated coefficient was the main parameter of interest. To account for unobserved heterogeneity, the model included random effects for player and referee, as well as the average skin color rating across referee's country of origin. While Group 5's analysis illustrates one possible approach, other groups may adopt different data preprocessing decisions, distributional assumptions (e.g., linear, logistic, or Poisson models), specification of fixed and random effects, or conduct the analysis in a Bayesian framework.

We use LLM to convert the methodology description into the schema we propose and generate the corresponding code script. We observe that most reports are structured to first describe the initial data analysis and then the changes made in the final analysis. To avoid testing LLMs' ability to reconstruct how analysis steps are modified, overturned, or carried over from the initial to the final analysis, we extract the descriptions relevant to the final analysis.

We send the text description of each team for LLM to generate the schema and then generate the code script from the schema. The code script is run by a separate LLM to validate for potential syntax errors.

To accelerate the evaluation of each script, we manually in-place replace the `lm()`/`glm()` command with `fixest::feols()`/`fixest::feglm()` (Bergé et al., 2026) for generalized linear model and the `lme4::glmer()` (Bates et al., 2015) function with `glmmTMB::glmmTMB()` (Brooks et al., 2017) for generalized mix-effect model after the scripts are generated but before the validation. These two functions are in-place replacements of the standard functions for estimating the relevant model with multi-threading in C++ (Bergé et al., 2026) and Template Model Builder (TMB) (McGillycuddy et al., 2025) for better performance. This replacement should not affect the estimation of the effect size. All the LLM-related simulations are conducted with Claude-Opus-4.5.

Across the 25 teams we reproduce, all team scripts were successfully executed after the validation phase. Notably, two scripts run successfully without any validation whereas the vast majority reached completion after two or three iterations. The most frequent correction involved resolving broken and overflow syntax in comments.

Figure 2 shows the comparison of the estimated effect size, in odd ratio, from our

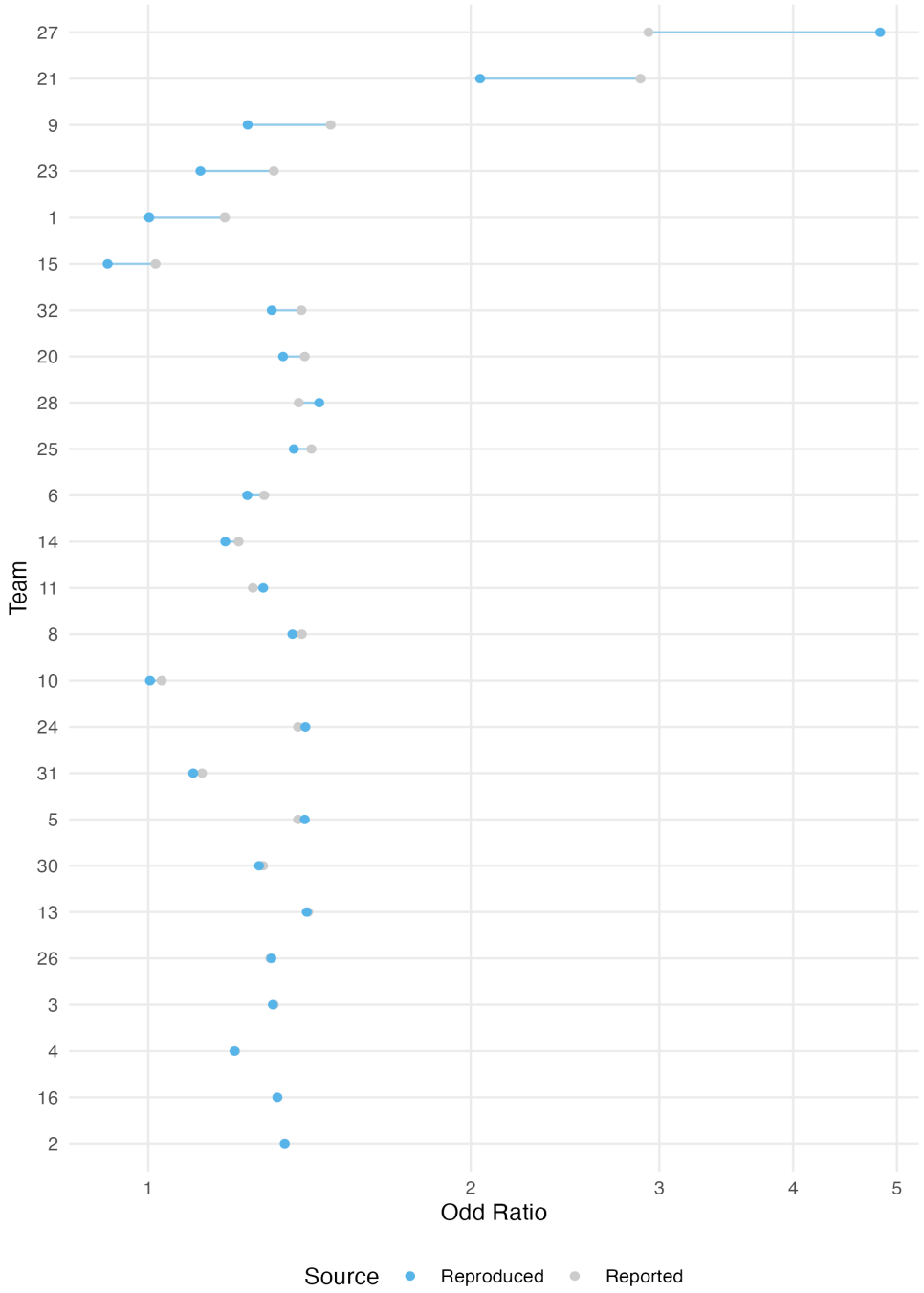


Figure 2. Comparison of the odd ratio of the effect of skin color on receiving red cards in football games from our schema-based reproduction (reproduced) vs. the effect size reported in Silberzahn et al. (2018) (reported). Teams are ordered by the absolute difference between the reproduced and reported odd ratio. The reproduction recovers the reported odd ratio closely for most teams; the largest deviations occur for Team 1, 9, 15, 21, 23, and 27.

workflow (simulated) and those reported in Silberzahn et al. (2018) (published). The odd ratio is well recovered for most teams, though six show a noticeably larger gap: Team 1, 9, 15, 21, 23, and 27. Looking closely at each team's report and generated script, the causes split into two stages of our pipeline, plus one case that has nothing to do with the LLM at all.

For Team 27, the report is genuinely ambiguous: the authors write that, "In addition to the independent variables specified in the initial approach, ... a variable for player position was included, as well as a variable for the league of the player." A statistician reading this might reasonably take it as shorthand for extending the model with interactions, consistent with how the other covariates entered the initial model, but the LLM read it literally and added only main effects. Team 9 fails for a similar reason at the same stage: the text names "referee country" as the covariate, yet the reported estimation table lists "player country" instead, and the LLM followed the text rather than reconciling it against the table. In both cases the schema itself is already wrong before any code is written, so we treat these as schema-extraction errors.

Team 1 and Team 23, by contrast, get the schema right but the code wrong. Both require reshaping the data from player-referee pairs to individual games, disaggregating win/loss/draw counts into game-level records. The schema correctly specifies this step, but the generated code fails to execute it as intended, and the resulting error passes through validation silently rather than triggering a repair.

Team 15's discrepancy is unrelated to either stage: it comes down to a difference in default contrast coding between R (our reproduction) and SPSS (the original analysis) — SPSS's effect coding compares each level to the grand mean, while R's default dummy coding compares to a reference level. Setting the `contrast` option in R to match recovers the reported odd ratio exactly, so this reflects a software convention rather than a modeling or LLM error.

That leaves Team 21, whose discrepancy we have not yet traced to a specific cause [TODO: resolve before submission]. With that exception, the pattern across the remaining five teams is that LLM error is split evenly between the schema-extraction and code-generation stages, while the sixth discrepancy is a reproduction artifact rather than a model failure.

3.1. *Exploring alternatives*

During the modelling process, researchers may consider a range of alternative specifications, such as different covariate and random effects structures, before arriving at a final model. This can be seen as a set of alternative decisions within the same modelling step, which fits naturally into the schema workflow described in Section 2.2. As an example, we use Team 5 for the above football study to demonstrate the use case. Team 5 uses a binomial generalized linear mixed model (GLMM) to estimate the effect of skin tone rating on the number of red cards received using a logit link function, but tests different

random effect structures:

$$\textbf{Model 1 (gm1):} \quad \begin{aligned} \text{logit}(p_{ijk}) &= \beta_0 + \beta_1 X_{ijk} + u_j + v_k \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad v_k \sim N(0, \sigma^2_{\text{ref}}) \end{aligned}$$

$$\textbf{Model 2 (gm2):} \quad \begin{aligned} \text{logit}(p_{ijk}) &= \beta_0 + \beta_1 X_{ijk} + u_j + v_{0k} + v_{1k} X_{ijk} \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad \mathbf{v}_k \sim N\left(\mathbf{0}, \mathbf{\Sigma}_{\text{ref}}\right) \end{aligned}$$

$$\textbf{Model 3 (gm3):} \quad \begin{aligned} \text{logit}(p_{ijkm}) &= \beta_0 + \beta_1 X_{ijkm} + u_j + v_k + w_{0m} + w_{1m} X_{ijkm} \\ u_j &\sim N(0, \sigma^2_{\text{player}}), \quad v_k \sim N(0, \sigma^2_{\text{ref}}), \quad \mathbf{w}_m \sim N\left(\mathbf{0}, \mathbf{\Sigma}_{\text{country}}\right) \end{aligned}$$

where p denotes the probability of a player receiving a red card, and X represents the player's skin tone rating. The subscripts i, j, k, and m index the individual observation, player, referee, and the referee's home country, respectively. The parameter β_0 represents the global baseline log-odds of receiving a red card, and β_1 estimates the fixed effect of a player's skin tone. The random effect include

- a random intercept for players, $u_j \sim N(0, \sigma^2_{\text{player}})$, in Model 1, 2, and 3,
- a random intercept for referee, $v_k \sim N(0, \sigma^2_{\text{ref}})$, in Model 1 and 3, an additional random slope for referee in Model 2, and
- a random intercept and slope for referee countries, $\mathbf{w}_m \sim N\left(\mathbf{0}, \mathbf{\Sigma}_{\text{country}}\right)$, in Model 3.

These alternatives are specified in a separate YAML file described in Section 2.2 as Listing 3. This file is then merged with the original schema to produce one separate schema for each alternative for code generation and validation. The validated scripts are then executed to obtain the estimated random effects in Table 1. This result matches with the random effects reported in the team report, subject to rounding errors. Similarly, the model fitting statistics can be extracted from the estimation for selecting the best fit.

Listing 3 Example of a YAML file specifying alternatives for the random effects structure in the many-analyst analysis of Group 5.

```
meta:
  type: alternatives
  step: specify_random_effects_structure
  fork: how to account for the non-nested cross-classified data structure
alternatives:
  - id: "gm1"
    path: "a random effect of player and referee"
    rationale: ""
  - id: "gm2"
    path: "a random effect of player and referee, and a random effect
      of skin-tone across referees"
    rationale: ""
  - id: "gm3"
    path: "a random effect of player, referee, and referees' countries
       of origin, and a random effect of skin tone across referees'
      countries of origin"
    rationale: ""
```

Table 1. Summary of random effect estimates for Group 5. The three models (gm1, gm2, and gm3) represent different specifications of the random effects structure to account for the non-nested cross-classified data structure. Cells are NA where a given random effect is not part of that model's specification.

Group	Term	gm1	gm2	gm3
Player	Intercept	0.3737	0.3738	0.2899
Referee	Intercept	0.1528	0.1932	0.0748
Referee	Referee rating	NA	0.0286	NA
Referee's country	Intercept	NA	NA	0.3990
Referee's country	Referee rating	NA	NA	0.0870

3.2. *Ablation: with and without an intermediate schema*

To assess whether the schema step itself improves the fidelity of the generated code, beyond what the underlying LLM can already achieve on its own, we conduct an ablation study comparing our schema-based pipeline against a direct pipeline that generates code straight from each team's methodology text. Both pipelines are applied to the same set of team descriptions, using the same underlying LLM (Claude-Opus-4.5) and the same script validation. The comparison of the two pipelines are shown in the Appendix. [TODO: main conclusion]

4. Discussion

Our results highlight a persistent gap between computational reproducibility and analytical understandability in modern data analysis workflows. While current reproducibility practices ensure that published results can be regenerated from shared data and code, as analyses grow in complexity and involve iterative decision-making processes, they are less of an ideal for readers to understand. Our proposed schema addresses this gap by introducing an intermediate representation that explicitly encodes analytical decisions to bridge between narrative descriptions and implementation details. When coupled with LLMs, this representation enables the extraction of analytical structure from written reports and its translation into analysis scripts, and supporting the exploration of alternative analytical choices.

Our approach differs in this respect from recent work that builds fully automated systems using multiple LLM agents to plan, generate, execute, and refine data analyses (Hu et al., 2024; Jansen et al., 2025). These systems rely on the same underlying capability of translating analytical specifications into code that our workflow uses, but they aim to automate the analytical decisions themselves, not just their translation into code. The failure modes we document in Section 3 suggest this is premature: even the comparatively narrower task of translating an already-specified decision into working code is not yet fully reliable. We take this as support for our position that the harder and more consequential part of a data analysis is the sequence of decisions a researcher makes, not the code that implements them, and that this part should remain explicit and in the hands of the analyst rather than be automated away.

To further examine how this workflow behaves in practice, we apply it to the many-analysts football study. This analysis allows us to identify errors in the original reports and to characterize failure modes of current LLMs to generate code for data science. Beyond validating the workflow itself, the schema-driven reanalysis catches a real inconsistency in a published report (Team 9's text-table mismatch, discussed in Section 3), which without an explicit account of the underlying decisions, would not have caught through computational reproducibility alone. Decomposing the largest discrepancies between our reproduced and reported effect sizes reveals that LLM error arises from two distinct stages of the pipeline, in roughly equal measure: mis-extraction of the schema from an ambiguous report, and correct schema extraction followed by incorrect code generation. This distinction matters for where future effort should be directed — schema-extraction errors call for better disambiguation strategies (e.g., using the confidence score ω_i to prompt for clarification, as introduced in Section 2), whereas code-generation errors call for improvements to the validation stage itself.

For the schema-extraction errors specifically, we find that LLMs require a fairly precise specification in order to generate the code correctly for the intended random effect structure. For an educated statistician, the phrase "a random effect of referee number across average rating" would typically imply both a random intercept and a random slope. However, we observe that LLMs interpret this specification literally as a random

slope only. This suggests the gap between the latest models and human analysts in specialized statistical knowledge (Sun et al., 2026).

For the code-generation errors, the LLM successfully corrects a range of implementation issues during validation, including errors caused by overflowing comments and other syntax issues. However, we also observe challenges in data reshaping tasks when the transformation requires changes in the unit of analysis. In the study, some teams decide to transform the data from the original aggregated player-referee dyad-level format to game-level format. This transformation requires disaggregating aggregated outcome counts (win, loss, and draw) into individual game records, changing the unit of analysis from dyads to matches. We also observe some difficulties in parsing birthdates. Since the LLM does not have access to the underlying raw data, due to privacy constraint, it must rely on trial and error when inferring the correct date formats ("DD-MM-YYYY."). This suggests that certain types of data analysis tasks may not be fully automatable in settings where the model cannot directly inspect the data, even if the data is theoretically available for computation.

While the schema requires a justification ρ_i for each decision, nothing in the format itself can guarantee that this justification is substantive rather than perfunctory; an analyst can satisfy the schema with a low-effort entry in the same way that a required code comment or commit message can be filled with boilerplate. The schema makes recording a decision's rationale cheap and structured, but, as with any documentation requirement, cannot enforce that authors use it thoughtfully. It remains an open question whether resolving $\mathcal{I}_i$ and $\mathcal{O}_i$ to concrete dataset variables ahead of time, rather than leaving this mapping for the LLM to infer at the point of code generation, meaningfully improves the reliability of the resulting code. [maybe we want a controlled experiment to compare code generation with or without pre-resolved input and output variables]

The reproducibility-understandability issue considered in this work is still at an early stage, some software decision choices are worth examining. We currently implement a prototype as an R package, although the understandability problem is broadly applicable to statistical analyses and is not associated to any particular programming language. Hence, we envision the implementation eventually evolving into a general purpose tool, such as a command-line utility, that can support analyses across different programming languages. In the current implementation, analytical decisions are recorded in a single YAML file. An alternative would be to record individual decisions in separate Markdown files. While YAML files provide an explicit structure for machine readability, markdown files offer greater flexibility for documenting decisions. Recording decisions in separate files would also allow files to be linked across time to understand how decisions evolve or change over the course of an analysis and can potentially support a broader research-management system if the decisions are properly attributed to individual researchers and maintains a record of the changes leading to the final analysis.

5. Acknowledgments

The implementation of the schema is available as an R package, `tines`, available on CRAN. The source code for reproducing the work reported in this paper is available in the Supplementary Materials and can be found at: https://github.com/huizezhang-sherry/paper-schema. The article is created using Quarto (Allaire et al., 2026) in R (R Core Team, 2026).

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.9) [Computer software]. https://doi.org/10.5281/zenodo.5960048

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bergé, L. R., Butts, K., & McDermott, G. (2026). Fast and user-friendly econometrics estimations: The R package fixest. *arXiv Preprint arXiv:2601.21749*. https://doi.org/10.48550/arXiv.2601.21749

Bertran, M., Fogliato, R., & Wu, Z. S. (2026). *Many AI Analysts, One Dataset: Navigating the Agentic Data Science Multiverse*. arXiv. https://doi.org/10.48550/arXiv.2602.18710

Brooks, M. E., Kristensen, K., van Benthem, K. J., Magnusson, A., Berg, C. W., Nielsen, A., Skaug, H. J., Maechler, M., & Bolker, B. M. (2017). glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R Journal*, *9*(2), 378–400. https://doi.org/10.32614/RJ-2017-066

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. *arXiv Preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374

Gao, R., & Xiao, S. C. (2026). *Nonstandard Errors in AI Agents*. arXiv. https://doi.org/10.48550/arXiv.2603.16744

Hu, S., Lu, C., & Clune, J. (2024, October). *Automated Design of Agentic Systems*. https://openreview.net/forum?id=t9U3LW7JVX

Huang, J., Wang, C., Zhang, J., Yan, C., Cui, H., Inala, J. P., Clement, C., & Duan, N. (2022). Execution-based Evaluation for Data Science Code Generation Models. In E. Dragut, Y. Li, L. Popa, S. Vucetic, & S. Srivastava (Eds.), *Proceedings of the Fourth Workshop on Data Science with Human-in-the-Loop (Language Advances)* (pp. 28–36). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.dash-1.5

Jansen, J. A., Manukyan, A., Khoury, N. A., & Akalin, A. (2025). Leveraging large language models for data analysis automation. *PLOS ONE*, *20*(2), e0317084. https://doi.org/10.1371/journal.pone.0317084

Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2026). A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–72.

Jun, E., Birchfield, M., De Moura, N., Heer, J., & Just, R. (2022). Hypothesis formalization: Empirical findings, software limitations, and design implications. *ACM Transactions on Computer-Human Interaction (TOCHI)*, *29*(1), 1–28.

Jun, E., Seo, A., Heer, J., & Just, R. (2022). *Tisane: Authoring statistical models via formal reasoning from conceptual and data relationships. CHI '22*, 116. https://doi.org/10.1145/3491102.3501888

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-T., Fried, D., Wang, S., & Yu, T. (2023). DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. *Proceedings of the 40th International Conference on Machine Learning*, 18319–18345. https://proceedings.mlr.press/v202/lai23b.html

Luo, A., Du, J., Tian, F., Xian, X., Specht, R., Wang, G., Bi, X., Fleming, C., Srinivasa, J., Kundu, A., Hong, M., & Ding, J. (2025). Can Agentic AI Match the Performance of Human Data Scientists? *2025 IEEE 10th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP)*, 206–210. https://doi.org/10.1109/CAMSAP66162.2025.11423949

McGillycuddy, M., Warton, D. I., Popovic, G., & Bolker, B. M. (2025). Parsimoniously fitting large multivariate random effects in glmmTMB. *Journal of Statistical Software*, *112*(1), 1–19. https://doi.org/10.18637/jss.v112.i01

Peng, R. D. (2011). Reproducible Research in Computational Science. *Science*, *334*(6060), 1226–1227. https://doi.org/10.1126/science.1213847

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://doi.org/10.32614/R.manuals

Shah, S. M. H., Hopfgartner, F., & Bleier, A. (2026). *Automating Computational Reproducibility in Social Science: Comparing Prompt-Based and Agent-Based Approaches*. https://doi.org/10.1145/3774905.3795485

Silberzahn, R., Uhlmann, E. L., Martin, D. P., Anselmi, P., Aust, F., Awtrey, E., Bahník, Š., Bai, F., Bannard, C., Bonnier, E., Carlsson, R., Cheung, F., Christensen, G., Clay, R., Craig, M. A., Dalla Rosa, A., Dam, L., Evans, M. H., Flores Cervantes, I., ... Nosek, B. A. (2018). Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science*, *1*(3), 337–356. https://doi.org/10.1177/2515245917747646

Simonsohn, U., Simmons, J. P., & Nelson, L. D. (2020). Specification curve analysis. *Nature Human Behaviour*, *4*(11), 1208–1214. https://doi.org/10.1038/s41562-020-0912-z

Steegen, S., Tuerlinckx, F., Gelman, A., & Vanpaemel, W. (2016). Increasing transparency through a multiverse analysis. *Perspectives on Psychological Science*, *11*(5), 702–712. https://doi.org/10.1177/1745691616658637

Sun, M., Han, R., Jiang, B., Qi, H., Sun, D., Yuan, Y., & Huang, J. (2026). LAMBDA: A Large Model Based Data Agent. *Journal of the American Statistical Association*, *121*, 1–13. https://doi.org/10.1080/01621459.2025.2510000

United Nations Development Programme. (2025). *Human development report 2025*. United Nations Development Programme. https://hdr.undp.org/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R.,

Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wrobel, J., Hector, E. C., Crawford, L., McGowan, L. D., Silva, N. da, Goldsmith, J., Hicks, S., Kane, M., Lee, Y., Mayrink, V., Paciorek, C. J., Usher, T., & Wolfson, J. (2024). Partnering With Authors to Enhance Reproducibility at JASA. *Journal of the American Statistical Association*, *119*(546), 795–797. https://doi.org/10.1080/01621459.2024.2340557